Namma Space – Technical Abstract (Round 1: 3D Reconstruction & Roaming)

Techfest IIT Bombay 2026–27 · Team Namma Space · Draft updated 2026-10-26 from the repository evidence. All performance and accuracy figures below come from recorded runs (files named in each table); none are estimates.

1. Problem and scope

Namma Space turns **ordinary phone photos or one video of an indoor space** into a **photo-textured 3D model that anyone can walk through in a web browser**, without installing software and without manual 3D modelling (OFF-14…16, OFF-23/24). Round 1 asks for an SOP, a reconstruction pipeline and a free-roaming web viewer. Everything runs **locally on the team's own laptops**, with no paid API or cloud service: the photo pipeline was built and evaluated on an Intel Core 5 120U (16 GB, integrated graphics, no CUDA) and uses an NVIDIA GPU when present (HP Victus, RTX 4050 Laptop 6 GB: COLMAP CUDA, Gaussian-splat training).

Own home (OFF-34). The team's room was captured with an iPhone 14 in **Polycam** (LiDAR) following our SOP and imported into the Studio, which repairs the scan, adds walking, labels, routes and measuring, and trains a **photoreal Gaussian-splat view** from the same capture (§3a). Two routes therefore exist: Polycam import (primary, LiDAR phones) and plain photos/video (any phone, §3).

2. System architecture

```
phone photos / video → Studio (local web app, 127.0.0.1) → capture check → worker process
  worker: prepare → COLMAP SfM (CPU) → undistort → OpenMVS dense → mesh → texture (GLB, embedded)
        → model frame (floor, up, start view, collision grid) → quality report → IF-13 scene package
viewer.html (three.js) ←— verified package (SHA-256 per file)      publish → static website / ZIP
```

- **Studio** (Python standard-library HTTP server + worker processes): project folders with JSON manifests, no database; one memory-heavy job at a time; every stage is resumable (completed stages are kept after a crash, a cancel or an out-of-memory failure).
- **Contracts**: the scene package (IF-13) states the model frame (right-handed, +Y up, floor at y = 0) and an explicit **scale status**. Photos alone cannot fix real-world scale, so the default is `unknown`; a user can add one known length later (no markers or tape required).
- **Viewer**: three.js, vendored (no CDN dependency); loads a light level of detail first, then full detail; every file is hash-checked before use; failures show a clear message instead of a broken scene.

3. Reconstruction pipeline

Stage	Tool / method	Why
Capture check	OpenCV / Pillow: sharpness (Laplacian variance), exposure, exact and near duplicates, ORB overlap graph; video: 3 frames/s, sharper of each pair (deterministic, FFmpeg)	Catch bad captures before minutes of processing
Camera poses	COLMAP 4.2.1 (CPU SIFT, exhaustive matching for photos, sequential for video, incremental mapping + second registration pass)	Robust, BSD-licensed; threads chosen from free RAM; guided matching disabled after it reached 9–10 GB
Dense geometry	OpenMVS 2.4.0 DensifyPointCloud (≤ 1600 px Preview, ≤ 2560 px Standard) on an evenly spread, sharpest-in-bin subset of ≤ 90 / ≤ 140 placed images	Best CPU-only MVS available to us; tracking needs dense video frames, surfaces do not
Surface	OpenMVS ReconstructMesh (a tested edge-length cleaning step was **not** adopted: it removed real flat surfaces)	Watertight-leaning surface from the dense points
Texture	OpenMVS TextureMesh → GLB; textures embedded as JPEG, local paths removed, seam levelling off (it blackened 17–34 % of texels in this build), neutral grey where no photo sees a face	Self-contained, portable, honest output
Model frame	Floor plane (RANSAC, scale-free tests on camera heights) or camera-orientation fallback; start view = the capture photo that overlooks most of the scene	The viewer opens at eye height, looking into the room
Quality	Registration rate, reprojection error, floor coverage, mesh fragmentation, textureless photos, unknown scale	Every warning is shown to the user in plain language

Two processing profiles: **Preview** (fast, 1600 px) and **Standard** (full-resolution photos with a memory cap, two levels of detail).

3a. Polycam route and photoreal view (own-home capture)

Stage	Method	Result on the team's room (iPhone 14, 218 keyframes)
Import	GLTF + Images export; zip-bomb and path checks; floating fragments removed; texture LOD ≤ 2048 px	50 k triangles, metric scale from the phone's tracking
Repair	Ceiling plane parallel to the floor, hanging debris removed, holes closed, backing sheets behind outer walls, texture gutters padded	Black holes and wavy ceiling fixed automatically (~4 s)
Walking maps	Floor found on the mesh; walkable map built on heights above the fitted floor plane (the export was 1.65° off level, which had hidden all clear floor)	Routes found with ≥ 0.28 m clearance
Camera poses	Phone (ARKit) poses refined by COLMAP and fitted back to the phone's metric frame	188 of 218 photos kept
Photoreal splat	gsplat 1.5.3, MCMC, 600 k Gaussians, 15 k steps started from points on the scanned mesh, per-photo pose and exposure correction, mesh depth guide, flat-wall prior; **~20 min** on an RTX 4050 Laptop (was 40 min, with a higher held-out score)	Held-out photos: **PSNR 22.6 → 26.3 dB, SSIM 0.789 → 0.858** over the first version
Line-up	Splat ↔ mesh: yaw search then ICP on solid Gaussians	55 % within 5 cm
Compression	Own SPZ v2 encoder; Gaussians under 2 % opacity dropped	**149 MB → 8.2 MB**; renders within 33–36 dB PSNR of the original

Each recipe change was kept only if it scored at least as well on photos held out of training.

4. Capture SOP (summary of docs/CAPTURE_SOP.md)

Route A, Polycam (used for our room): LiDAR mode; walk slowly along every wall pointing the phone straight at it from 1–1.5 m (a wall seen only from across the room smears in the photoreal view); one lap tilted down for the floor, one up for the ceiling line; go around large furniture; export GLTF + Images. **Route B, photos:** Lights on; main 1× lens; JPEG ("Most Compatible") on iPhone; walk along the walls facing into the room, one step (~0.5 m) between photos, about two-thirds overlap; floor in the lower third of every photo; a second, lower lap around large furniture; 3–4 photos while passing each doorway; always include an edge or object when photographing plain walls, glass or mirrors; 60–150 photos per room, or one slow 1–3 minute video. The Studio's capture check enforces the limits (10–300 photos, one video, JPEG/PNG, ≥ 480 px) and explains every warning.

5. Web viewer

The viewer opens on the **photoreal view** (Spark 2.4.0 renders the splat inside the same three.js scene); the mesh stays underneath for collision, labels, routes and measuring. A **measuring tool** (M) gives chained lengths with across/height components, level/upright snapping and closed-shape areas (m, cm or ft-in); a **mini-map** shows a top-down plan of the room with labels, the route and the viewer's position. First-person controls: click to capture the mouse (pointer lock) or drag to look; W A S D / arrows to walk; Shift to run; F toggles fly mode; R resets to the start view; [ ] jump through the original photo viewpoints. Walking keeps the camera at the capture eye height inside the free space connected to where the photos were taken (reconstructed furniture and walls block; floater specks do not). Floor areas the photos did not reconstruct are shown by a faint, labelled guide grid instead of a void or invented geometry. Place labels can be added by clicking on the model and found by search (jump transition). Walking paths are shown only as **unverified** guidance (Round 2 scope).

6. Evaluation on public datasets

Ground truth: ETH3D laser scans and DSLR poses; ARKitScenes ARKit trajectory, LiDAR mesh and annotated furniture boxes; TUM motion capture. Development scenes were used for tuning; **held-out scenes were processed once with the frozen configuration** (protocol `phase13_artifacts/PROTOCOL_PHASE13.md`). Criteria are the Phase 12 pre-registered ones, unchanged.

Scene (role, profile, config)	Photos placed	Camera error median	Dense accuracy median	Completeness @5 cm (dense / mesh)	Floor IoU	Routes OK (false-unreachable)	Time · peak RAM
arkit_41069025 (held-out, preview, frozen)	35/300	12.8 mm	8.9 mm	0.14 / 0.16	0.38	1/12 (11)	19.0 min · 2.2 GB
arkit_41142280 (held-out, preview, frozen)	80/175	14.8 mm	21.7 mm	0.21 / 0.22	0.10	not measurable	22.8 min · 3.0 GB
eth3d_delivery_area (held-out, preview, frozen)	44/44	14.8 mm	13.0 mm	0.61 / 0.71	0.61	4/12 (8)	28.6 min · 4.0 GB
eth3d_office (held-out, preview, frozen)	26/26	4.8 mm	6.8 mm	0.30 / 0.40	0.00	0/12 (12)	7.5 min · 1.9 GB
eth3d_office (held-out, standard, frozen)	26/26	5.3 mm	6.0 mm	0.33 / 0.38	0.00	0/12 (12)	34.2 min · 6.8 GB
eth3d_delivery_area (dev, standard, earlier)	44/44	13.0 mm	11.0 mm	0.66 / 0.76	0.62	4/12 (8)	58.5 min · 11.3 GB
eth3d_kicker (dev, standard, earlier)	30/31	7.7 mm	6.3 mm	0.49 / 0.59	0.34	0/12 (12)	51.1 min · 7.6 GB
eth3d_kicker (dev, standard_fullres, earlier)	30/31	7.5 mm	6.4 mm	0.50 / 0.60	0.35	0/12 (12)	51.3 min · 6.9 GB
eth3d_pipes (dev, preview, earlier)	14/14	5.1 mm	8.7 mm	0.29 / 0.45	0.00	0/12 (12)	3.8 min · 1.9 GB
tum_fr3_office_sample (dev, preview, earlier)	60/61	7.4 mm	–	– / –	–	not measurable	4.8 min · 0.6 GB
tum_fr3_office_sample (dev, preview, earlier)	61/61	8.6 mm	–	– / –	–	not measurable	5.7 min · 0.6 GB

Failed runs (kept on record, not hidden): arkitscenes-41125700-dev (preview, stopped in mesh): Unexpected error in Building the surface mesh: [WinError 2] The system cannot …; arkitscenes-41125700-dev (preview, stopped in frame): Unexpected error in Finding the floor and orientation: roam grid too large …; eth3d-delivery-area-held-out (standard, stopped in texture): Painting photo textures onto the mesh took longer than allowed.; eth3d-kicker-dev (standard, stopped in sfm): Placing cameras (structure from motion) failed: colmap_exhaustive_matcher …; eth3d-pipes-dev (preview, stopped in undistort): Correcting lens distortion failed: colmap_image_undistorter failed with exit ….

Config: *frozen* = configuration frozen before the held-out runs; *final* = the submitted code (frozen + post-freeze face cap D13-14); *earlier* = development configuration.

Criteria (Phase 12, unchanged): all photos placed; camera error ≤ 50 mm; dense accuracy ≤ 50 mm; dense completeness @5 cm ≥ 0.50; ≥ 1 route with 0 collisions and 0 false-unreachable pairs; preview ≤ 30 min, standard ≤ 90 min; peak RAM ≤ 12 GB. Per-criterion verdicts: `docs/RESULTS_SCENES.md`; gates A–F: `docs/RESULTS_MATRIX.md`.

Viewer frame time on the laptop's Intel iGPU (real key / mouse events): visible Chrome window (visible_kicker_standard): median 16.7 ms / P95 16.9 ms at 3.27 M textured triangles; headless Chrome, 11 records on 9 packages: median 16.7–33.3 ms (not a visible-window benchmark).

7. Limitations and next steps

- **Own home via Polycam.** The own-home model uses Polycam's LiDAR mesh (third-party app; our code repairs it and adds the photoreal view, walking, routes and measuring). Splat quality is limited by capture coverage: a wall the phone only saw from far away smears up close; the SOP now asks for wall passes at 1–1.5 m.
- **Photoreal training needs an NVIDIA GPU** (~20–25 min on an RTX 4050 Laptop for 218 photos); without one, the mesh view is used.
- **Completeness depends on the room.** Textured, cluttered spaces reconstruct well (held-out delivery_area 0.61); plain white walls, desks and dark floors stay partly missing (held-out office 0.30–0.33). Missing floor is shown by a

labelled guide grid and reported, never filled with invented geometry.

- **Fast handheld video fails.** On both held-out iPad videos CPU structure-from-motion placed only 12–46 % of the frames; the result is labelled "poor". The SOP asks for photos or a slow video.
- **Scale.** Photos do not fix real-world size: the model is labelled "scale unknown" until the user enters one known length. The fallback assumption (1.4 m phone height) was off by −5 % to +90 %, so it is never shown as metric.
- **Navigation is not validated.** No scene meets the route criteria (best: 4 routes, 0 collisions, 8 pairs missed); walking paths are shown only as *unverified* (Round 2: floor completion with human verification, ARCH-030).
- **Compute (photo route).** CPU only: Preview 4–40 s per image, Standard 80–100 s per 24 MP photo. A 44-photo hall at Standard needed 11.2 GB and exceeded the time limit (failed, kept); Standard is now capped at 4 M faces and the Studio advises Preview first above 40 photos.
- **Licences.** OpenMVS is AGPL-3.0 (used unmodified as an external program, never bundled); ETH3D and ARKitScenes are non-commercial: their derived models are not published.

8. Credits and licences

COLMAP 4.2.1 (BSD-3-Clause), OpenMVS 2.4.0 (AGPL-3.0; used as an unmodified external program, never bundled), three.js 0.186.1 (MIT), FFmpeg (external tool), NumPy / SciPy / Shapely (BSD), OpenCV (Apache-2.0), Pillow (MIT-CMU), jsonschema (MIT), psutil (BSD), PyTorch (BSD-3-Clause), gsplat 1.5.3 (Apache-2.0), Spark 2.4.0 (MIT). Polycam (third-party capture app; its exports are the input of the import route). Datasets: ETH3D (Schöps et al., CVPR 2017; CC BY-NC-SA 4.0), ARKitScenes (Baruch et al., NeurIPS 2021 D&B; Apple non-commercial licence), TUM RGB-D (Sturm et al., IROS 2012; CC BY 4.0). Full notices: THIRD_PARTY_NOTICES.md.